Individual Contribution Report

MiniOS Project – Operating Systems Simulation

Student Name:	ANNIKKA C. CAMPOREDONDO
Group:	MiniOS Omega (1063)
Assigned Role:	File Management Developer

1. What did I contribute to the project?

I was responsible for the **file management module** of MiniOS (`filesystem.py`). This module simulates a file system entirely in memory and provides the commands users type to work with files and folders. I designed the data structure behind it, implemented my assigned commands, added input validation and error messages, and tested them with the rest of the shell in `main.py`.

2. What features did I personally develop?

Command	What it does
`dir`	Lists the contents of the current directory, showing <DIR> for folders and the size in bytes for files. Prints “(empty)” for empty folders and sorts entries alphabetically.
`mkdir <folder>`	Creates a new directory. Rejects names that already exist.
`create <file>`	Creates a new empty file. Rejects names that already exist.
`read <file>`	Displays a file’s contents, or “(empty file)” if it has none.
`delete <name>`	Deletes a file or an empty directory. Refuses to delete non-empty directories so data is not lost by accident.

Shared design work: the file system is stored as nested Python dictionaries, where a **folder is a dict** and a **file is a string**. A `cwd` list tracks the current path, and a helper, `_current_dir()`, walks that path to return the active folder. All commands use this helper, which keeps the code short and consistent.

3. What problems did I encounter?

- Deciding how to represent files and folders so that every command could use the same structure.
- Telling files and folders apart, so that `read` does not try to print a folder and `write`/`read` do not treat a folder as a file.
- Preventing duplicate names, which would silently overwrite existing files or folders.
- Avoiding accidental data loss when deleting a folder that still contains files.
- Handling missing or empty arguments so the shell does not crash on input such as `mkdir` with no name.

4. How did I solve those problems?

- Used one nested-dictionary tree with a `_current_dir()` helper, so no command needs its own path logic.
- Used `isinstance(item, dict)` to distinguish folders from files before acting on an entry.

- Checked whether the name already exists in the current folder before `mkdir` and `create`, and print “Name already exists.” instead of overwriting.
- Made `delete` refuse non-empty directories with a clear error message.
- Added a usage message to each command (for example `Usage: mkdir <folder>`) when arguments are missing, and relied on the shell’s error handling in `main.py` as a safety net.

5. What did I learn from this project?

- How an operating system’s file system can be modelled with a tree data structure, and how the current working directory is tracked.
- The importance of input validation and clear error messages in command-line tools.
- How to split a program into modules (users, files, processes, memory) so that each team member can work independently and the parts still fit together.
- Working as a team on shared code and agreeing on a common command format.

6. Evidence of my contribution

Source file: `filesystem.py` (commands `dir`, `mkdir`, `create`, `read`, `delete`). Below is a sample session that exercises my commands, including the error cases:

```
MiniOS> dir
Directory of \
  (empty)
MiniOS> mkdir docs
Directory created.
MiniOS> create notes.txt
File created.
MiniOS> mkdir docs
Error: Name already exists.
MiniOS> write notes.txt Hello MiniOS
File saved.
MiniOS> dir
Directory of \
  <DIR>    docs
  12 B     notes.txt
MiniOS> read notes.txt
Hello MiniOS
MiniOS> read docs
Error: File does not exist.
MiniOS> cd docs
MiniOS> create a.txt
File created.
MiniOS> cd ..
MiniOS> delete docs
Error: Directory is not empty.
MiniOS> delete notes.txt
Deleted.
MiniOS> delete ghost
Error: File or directory does not exist.
MiniOS> dir
Directory of \
  <DIR>    docs
```

GitHub commits / pull requests: _____

Screenshots: attach screenshots of the commands running in the MiniOS shell.